

Fairy's Card Game

Generated by Doxygen 1.8.17

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 Cards Struct Reference	5
3.1.1 Detailed Description	5
3.2 Players Struct Reference	5
3.2.1 Detailed Description	6
4 File Documentation	7
4.1 cardGame.c File Reference	7
4.1.1 Detailed Description	9
4.1.2 Function Documentation	9
4.1.2.1 applyCardEffect()	9
4.1.2.2 dealCards()	9
4.1.2.3 initializeDeck()	10
4.1.2.4 initializePlayers()	10
4.1.2.5 main()	10
4.1.2.6 playGame()	11
4.1.2.7 playingField()	11
4.1.2.8 randomstart()	12
4.1.2.9 removePlayer()	12
4.1.2.10 shuffledeck()	12

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Cards		
	Cards in the deck	5
Players		
	Players in the game	5

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

cardGame.c	
Implementation of a card game	7

Chapter 3

Class Documentation

3.1 Cards Struct Reference

represents cards in the deck

Public Attributes

- char [v](#) [3]
rank/value of the card
- char [s](#) [10]
suit/symbol of the card
- bool [faceUp](#)
If true the card is faceUp, if false the card is faceDown.

3.1.1 Detailed Description

represents cards in the deck

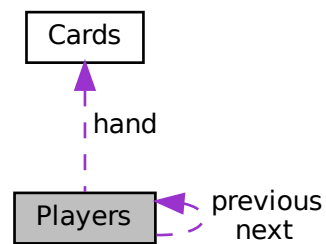
The documentation for this struct was generated from the following file:

- [cardGame.c](#)

3.2 Players Struct Reference

represents players in the game

Collaboration diagram for Players:



Public Attributes

- char [name](#) [50]
name of the player
- int [lifePoints](#)
lifepoints of the player
- [Cards](#) [hand](#) [2]
a hand of two cards
- struct [Players](#) * [previous](#)
pointer to the previous player
- struct [Players](#) * [next](#)
pointer to the next player

3.2.1 Detailed Description

represents players in the game

The documentation for this struct was generated from the following file:

- [cardGame.c](#)

Chapter 4

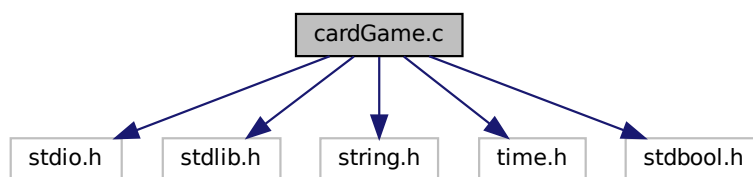
File Documentation

4.1 cardGame.c File Reference

Implementation of a card game.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <stdbool.h>
```

Include dependency graph for cardGame.c:



Classes

- struct [Cards](#)
represents cards in the deck
- struct [Players](#)
represents players in the game

Macros

- `#define RESET "\033[0m"`
It resets terminal text color to default.
- `#define RED "\033[1;31m"`
it sets terminal text color to red for warning messages
- `#define GREEN "\033[1;32m"`
it sets terminal text color to green to ask players whether they want to reveal their faceDown card or not
- `#define MAGENTA "\033[35m"`
it sets terminal text color to magenta for error messages
- `#define CYAN "\033[36m"`
it sets terminal text color to cyan
- `#define YELLOW "\033[33m"`
it sets terminal text color to yellow to indicate players' information and turns

Typedefs

- `typedef struct Players Players`
represents players in the game

Functions

- `void initializeDeck (Cards *Deck)`
initilaze a 40-card deck All cards are set to faceDown at the beginning
- `void shuffledeck (Cards *Deck, int Deck_size)`
shuffle the deck randomly
- `void dealCards (Cards *Deck, Players *player, int number_of_players)`
deals two cards to every player
- `void playingField (int number_of_players, const Players *player, int *lifePointsinField)`
demonstrate playing field and players' information
- `void initializePlayers (Players *player, int number_of_players)`
initialize players with names, two lifepoints and two cards (one faceDown and one faceUp)
- `void removePlayer (Players *player, int *active)`
remove eliminated players from the circular list
- `void applyCardEffect (Players *player, Cards card, int *active, int *lifePointsinField)`
- `int randomstart (int number_of_players)`
- `void playGame (Players *player, int number_of_players, Cards *Deck, int Deck_size)`
- `int main ()`

Variables

- `const char * symbols [] = {"Hearts", "Diamonds", "Clubs", "Spades"}`
Array for symbols of cards (Hearts, Dioamands, Clubs, Spades)
- `const char * values [] = {"1", "2", "3", "4", "5", "6", "7", "J", "Q", "K"}`
Array for values of cards (1-7, J, Q, K)

4.1.1 Detailed Description

Implementation of a card game.

This code shows a card game which players take random turns and use special card effects to each other Card effects can either increase or decrease players' life-points The goal is to be the last alive player in the game to win

Features include: -Player initialization, elimination and management -Deck initialization, shuffling and dealing - Special card effects and playgame mechanics

4.1.2 Function Documentation

4.1.2.1 applyCardEffect()

```
void applyCardEffect (
    Players * player,
    Cards card,
    int * active,
    int * lifePointsinField )
```

applies the effect of a special card different cards have different effects: '1' : lose 1 Life point and put it on the field '7' : forces the next player to reveal faceDown card 'J' : gives 1 life point to the previous player 'Q' : gives 1 life point to the second next player 'K' : takes all life points from playing field

Parameters

<i>player</i>	the player who played the card
<i>card</i>	the card being played
<i>active</i>	pointer to the number of active players
<i>lifePointsinField</i>	Pointer to the life points in field

4.1.2.2 dealCards()

```
void dealCards (
    Cards * Deck,
    Players * player,
    int number_of_players )
```

deals two cards to every player

first card is dealt faceDown and other one is faceUp

Parameters

<i>Deck</i>	pointer to shuffled deck of cards
<i>player</i>	array of players who receive dealing cards
<i>number_of_players</i>	total number of players in the game

4.1.2.3 initializeDeck()

```
void initializeDeck (
    Cards * Deck )
```

initilize a 40-card deck All cards are set to faceDown at the beginning

Parameters

<i>Deck</i>	pointer to cards array to be initilized
-------------	---

4.1.2.4 initializePlayers()

```
void initializePlayers (
    Players * player,
    int number_of_players )
```

initialize players with names, two lifepoints and two cards (one faceDown and one faceUp)

Players are linked in a circular list.

Parameters

<i>player</i>	array of players to be initilized
<i>number_of_players</i>	total number of players in the game

4.1.2.5 main()

```
int main ( )
```

The main function of the code this function asks for the desired number of players by setting a condition (number of players must be between 2 and 20) and initilizes the game This part also sets up the deck of the cards Moreover, it invokes the playGame functions by implying all the rules of the game during its execution

Parameters

<i>number_of_players</i>	the total players present in the game
<i>player</i>	An array containing all the information of each player
<i>Deck</i>	This is also an array representing the deck of cards which are used in the game Last step, it returns 0 as a sign of a successful execution

4.1.2.6 playGame()

```
void playGame (
    Players * player,
    int number_of_players,
    Cards * Deck,
    int Deck_size )
```

starts the game shuffles the deck and deals cards to the present players function keep running until someone wins

Parameters

<i>player</i>	the array of all players
<i>number_of_players</i>	this represents the total number of the players
<i>Deck</i>	this is the deck of the cards @oaram Deck_size: this indicates the total number of cards present in the deck

4.1.2.7 playingField()

```
void playingField (
    int number_of_players,
    const Players * player,
    int * lifePointsinField )
```

demonstrate playing field and players' information

expresses remaining players and their life-points

Parameters

<i>lifePointsinField</i>	pointer to life points of the playing field
<i>player</i>	array of players in the game
<i>number_of_players</i>	total number of players in the game

4.1.2.8 randomstart()

```
int randomstart (
    int number_of_players )
```

Chooses a random player to start the game

Parameters

<i>number_of_player</i>	total number of players
-------------------------	-------------------------

Returns

index of the player who will start

4.1.2.9 removePlayer()

```
void removePlayer (
    Players * player,
    int * active )
```

remove eliminated players from the circular list

updates previous and next players and active count is reduced

Parameters

<i>player</i>	pointer to the player to be eliminated
<i>active</i>	pointer to the total number of active players

4.1.2.10 shuffledeck()

```
void shuffledeck (
    Cards * Deck,
    int Deck_size )
```

shuffle the deck randomly

Swap cards in random with Fisher_Yates algorithm

Parameters

<i>Deck</i>	pointer to cards array
<i>Deck_size</i>	is number of cards in the deck